

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5

6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I **A Mohammad Naheed(192525336)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Mohammad Naheed (192525336)

1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

1. Recommended file organization: B+ tree (indexed) organization

For **10 million product records**, I would recommend an **indexed sequential file organization using a B+ tree index** on the primary product key (e.g., ProductID).

Why this choice?

- **Fast product lookup:** A B+ tree provides approximately $O(\log n)$ search, so finding one product remains fast even with 10 million records.
- **Supports range queries:** Queries such as “products priced between ₹500 and ₹1,000” or “all products in category Electronics” can be handled efficiently if appropriate secondary indexes are created.
- **Efficient insertion/deletion:** E-commerce catalogs change frequently. B+ trees can insert and delete records without requiring the entire file to be reorganized.
- **Good disk utilization:** B+ tree nodes are designed around disk/page access, minimizing expensive disk I/O.
- **Scalability:** The structure remains practical as the catalog grows beyond 10 million records.

Cost analysis

Assume, for illustration:

- 10 million records
- Average record size = **1 KB**
- Data size = approximately **10 GB**
- Disk page size = **4 KB**
- Each index entry requires roughly **32 bytes**

Organization	Search cost	Insertion cost	Storage overhead	Suitability
Heap/unordered	$O(n)$	Very low	Very low	Poor for frequent product searches
Sequential/sorted	$O(\log n)$ with binary search	High	Low	Moderate
Hash file	$O(1)$ average	Low	Moderate	Excellent for exact lookup, poor for range queries
B+ tree indexed	$O(\log n)$	$O(\log n)$	Moderate	Best overall

A rough B+ tree index calculation illustrates the scale. If each 4 KB page holds about 100 index entries, the first-level index would need around:

$$10,000,000 / 100 = 100,000 \text{ pages}$$

Higher levels reduce this dramatically: roughly 1,000 pages at the next level and about 10 pages at the next. Thus, the tree would typically require only **a few page accesses** to locate a product, rather than scanning millions of records.

The index itself might be on the order of **hundreds of MB**, depending on the actual key and pointer sizes. Compared with roughly **10 GB of product data**, this is a relatively small storage cost.

Conclusion

B+ tree indexed organization is the best general-purpose choice because it balances **search speed, insertion/deletion performance, range-query capability, and storage cost**. A hash organization could be cheaper and faster for *only* exact ProductID lookups, but an e-commerce system usually needs category, price, brand, and other range/filter queries, making B+ trees more versatile.

2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

1. Indexing Strategy for a Banking Transaction System

For a banking system, transactions are commonly retrieved using **account number** and a **date range**. A **composite B+ tree index** is therefore the most appropriate primary index.

1. Primary composite index

Create a B+ tree index on:

(account_number, transaction_date)

For example:

```
CREATE INDEX idx_transaction_account_date  
ON Transactions(account_number, transaction_date);
```

This index is effective for queries such as:

```
SELECT *
```

```
FROM Transactions
```

```
WHERE account_number = 12345
```

```
AND transaction_date BETWEEN '2026-01-01' AND '2026-01-31';
```

The database first locates the account using `account_number`, then efficiently scans only the relevant date range.

2. Why a B+ tree?

- **Fast equality search:** Quickly finds all records belonging to an account.
- **Efficient range search:** B+ trees are well suited to date-range queries.
- **Ordered dates:** Leaf nodes are linked, making sequential retrieval of transactions within a date range efficient.
- **Scalable:** Performance remains good even with millions or billions of transactions.

3. Optional secondary indexes

Depending on the application's queries, additional indexes can be created:

Query requirement	Recommended index
Account + date range	B+ tree (<code>account_number</code> , <code>transaction_date</code>)
Transaction ID lookup	Hash or B+ tree on <code>transaction_id</code>
Transactions by date across all accounts	B+ tree on <code>transaction_date</code>
Fraud/status searches	B+ tree or bitmap index, depending on workload

A **hash index** is useful for exact lookups such as `transaction_id = 987654`, but it is **not appropriate for date ranges**, because hashing destroys the ordering needed for efficient range scans.

4. Additional consideration: very large transaction tables

If the transaction table becomes extremely large, transactions can also be **partitioned by date** (for example, monthly or quarterly). Within each partition, the (`account_number`, `transaction_date`) B+ tree index can be maintained.

Conclusion

The recommended design is:

Partition transactions by date (if the table is very large) + B+ tree composite index on (`account_number`, `transaction_date`) + optional indexes for other frequent access patterns.

This provides fast retrieval for the most important query pattern—"give me all transactions for this account during this date range"—while keeping indexing overhead manageable.

3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Multi-Level Indexing Solution

For a healthcare system, patient records need to be accessed efficiently using **PatientID** as well as **doctor name**. A suitable design is a **primary index on PatientID with a secondary index on DoctorName**.

1. Primary index — PatientID

Create a **B+ tree primary index** on PatientID.

```
CREATE UNIQUE INDEX idx_patient_id  
ON PatientRecords(PatientID);
```

- PatientID is unique, so it provides direct identification of a patient's record.
- A B+ tree gives approximately $O(\log n)$ search time.
- It supports efficient insertion and deletion as patient records change.
- Since PatientID is the primary access key, this should be the main index.

2. Secondary index — DoctorName

Create a **secondary B+ tree index** on DoctorName.

```
CREATE INDEX idx_doctor_name  
ON PatientRecords(DoctorName);
```

This allows queries such as:

```
SELECT *
```

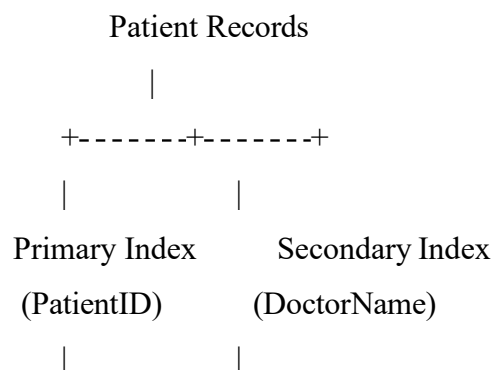
```
FROM PatientRecords
```

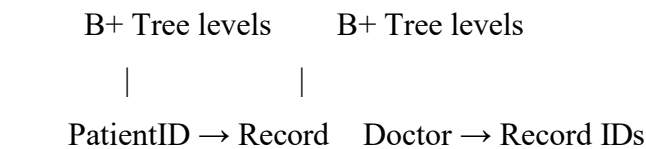
```
WHERE DoctorName = 'Dr. Kumar';
```

Because multiple patients may be treated by the same doctor, this is a **non-unique secondary index**. Each doctor-name entry points to multiple patient records.

3. Multi-level structure

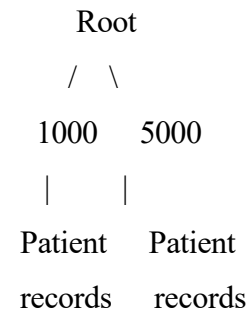
Conceptually:



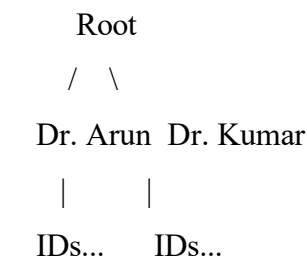


For example:

PatientID Index



DoctorName Index



4. Optional third-level indexing

If doctors commonly search patients by **doctor + appointment date**, a composite index can be added:

```
CREATE INDEX idx_doctor_date
ON PatientRecords(DoctorName, AppointmentDate);
```

This avoids scanning all patients belonging to a doctor when only a particular date range is required.

Cost and performance

Index	Purpose	Typical search
B+ tree on PatientID	Individual patient retrieval	$O(\log n)$
B+ tree on DoctorName	Find patients of a doctor	$O(\log n + k)$
B+ tree on (DoctorName, AppointmentDate)	Doctor/date queries	$O(\log n + k)$

Here, **k** is the number of matching records.

Conclusion

The recommended multi-level indexing strategy is:

Primary **B+** **tree** **index** → PatientID

Secondary **B+** **tree** **index** → DoctorName

Optional composite B+ tree → (DoctorName, AppointmentDate)

This provides fast access through both major search paths while avoiding the high cost of scanning the entire patient-record table.

4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

1. B-Tree vs B+ Tree for Shipment Records

For a logistics company with **frequent dynamic insertions and deletions**, a **B+ Tree is generally more suitable than a B-Tree**.

Comparison

Feature	B-Tree	B+ Tree
Data stored	Internal and leaf nodes	Leaf nodes only
Search	$O(\log n)$	$O(\log n)$
Insertion	$O(\log n)$	$O(\log n)$
Deletion	$O(\log n)$	$O(\log n)$
Range queries	Less efficient	Very efficient
Sequential access	Moderate	Excellent
Fan-out	Lower	Higher
Disk I/O	Relatively higher	Lower

Why B+ Tree is preferable

A shipment database may contain queries such as:

SELECT *

FROM Shipments

WHERE ShipmentDate BETWEEN '2026-08-01' AND '2026-08-25';

B+ Trees store actual record pointers at the **leaf level**, and their leaf nodes are linked. Therefore, once the first matching shipment is found, the remaining shipments in the date range can be retrieved sequentially with minimal additional tree traversal.

For dynamic operations:

- **Insertion:** New shipment entries are inserted into the appropriate leaf. If a node becomes full, it is split while maintaining the tree's balance.
- **Deletion:** Entries can be removed and nodes merged or redistributed when necessary.
- **Balanced structure:** Both operations maintain approximately the same logarithmic height.
- **High fan-out:** Internal nodes contain many keys and pointers, keeping the tree relatively shallow and reducing disk accesses.

Cost consideration

If there are **N shipment records**, both structures provide approximately:

- Search: **$O(\log N)$**
- Insertion: **$O(\log N)$**
- Deletion: **$O(\log N)$**

However, B+ Trees usually have a **higher fan-out** because internal nodes store only keys and child pointers. This produces a smaller tree height and therefore fewer disk I/O operations.

Conclusion

Choose a B+ Tree. Although both B-Trees and B+ Trees efficiently support dynamic insertion and deletion, B+ Trees are better suited to logistics systems because they combine efficient updates with **fast range searches and sequential retrieval**—both common requirements when tracking shipments by date, location, status, or delivery time.

5.Design a hashing scheme for an HR system with 5000 employees.

Evaluate static hashing vs. extendible hashing for your scenario.

1. Hashing Scheme for an HR System

For an HR system with **5,000 employees**, suppose each employee has a unique EmployeeID and records are frequently retrieved using this ID.

1. Proposed hashing scheme

Use **EmployeeID** as the search key and apply a hash function such as:

$$h(\text{EmployeeID}) = \text{EmployeeID} \text{ } 503$$

Here, **503** is a prime number close to the desired number of buckets.

If each bucket can hold about **10 records**, the initial capacity is:

$$503 \times 10 = 5,030 \text{ records}$$

This provides a small amount of spare capacity for 5,000 employees.

Example:

EmployeeID = 12567

$$\begin{aligned} h(12567) &= 12567 \bmod 503 \\ &= 2 \end{aligned}$$

Therefore, the record is placed in **bucket 2**.

For collisions, use **overflow buckets/chaining** rather than simply overwriting existing records.

2. Static Hashing

With static hashing, the number of buckets is fixed when the file is created.

Advantages:

- Simple implementation.
- Very fast exact-match searches: approximately **O(1)** average time.
- Low management overhead.
- Suitable when the employee count remains close to 5,000.

Disadvantages:

- If the company grows significantly, buckets become increasingly full.
- More collisions and overflow chains can occur.
- Increasing the number of buckets requires **rehashing**, which can be expensive for a large HR database.

For example, if the company grows from 5,000 to 15,000 employees, the original 503 buckets would be inadequate and substantial reorganization could be required.

3. Extendible Hashing

Extendible hashing uses a **directory of bucket pointers** and dynamically splits buckets when they become full.

Directory

```
|
+---- 00---- > Bucket A
+---- 01---- > Bucket B
+---- 10---- > Bucket C
```

+---- 11---- > Bucket D

When a bucket overflows, only the affected bucket is split, and the directory may be expanded.

Advantages:

- Automatically adapts as employees are added or removed.
- Significantly reduces long overflow chains.
- No complete-file rehashing is normally required.
- Provides approximately **O(1)** average lookup, insertion, and deletion.

Disadvantages:

- More complex than static hashing.
- Requires additional directory storage.
- Bucket splitting can introduce additional I/O.

4. Static vs. Extendible Hashing

Factor	Static Hashing	Extendible Hashing
Implementation	Simple	More complex
Exact lookup	O(1) average	O(1) average
Growth handling	Poor	Excellent
Overflow handling	Overflow buckets	Dynamic bucket splitting
Storage overhead	Low	Moderate
Reorganization	May require rehashing	Usually local
Best when	Size is stable	Size changes frequently

Recommendation

For an HR system with **exactly around 5,000 employees and little expected growth**, **static hashing with about 503 buckets** is sufficient and economical.

However, if the organization is expected to grow or shrink substantially, I would choose **extendible hashing**. HR databases tend to change over time as employees join, leave, and records are added, so **extendible hashing is the safer long-term design**.

Final choice: **Extendible hashing** if scalability is an important requirement; otherwise, static hashing is simpler and cheaper for a stable 5,000-employee database.

6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

1. Combined Index Structure for an Airline Reservation System

The system has two main access patterns:

- **Point query:** Find a flight using FlightNumber.
- **Range query:** Find flights within a particular FlightDate range.

The best solution is to use **two B+ tree indexes**, with a composite index as an optional optimization.

1. B+ Tree on FlightNumber

Create a unique B+ tree index:

```
CREATE UNIQUE INDEX idx_flight_number  
ON Flights(FlightNumber);
```

This provides fast point lookups:

```
SELECT *  
FROM Flights  
WHERE FlightNumber = 'AI202';
```

The lookup takes approximately **$O(\log N)$** time and is highly efficient even when the database contains millions of flights.

2. B+ Tree on FlightDate

Create a second B+ tree index:

```
CREATE INDEX idx_flight_date  
ON Flights(FlightDate);
```

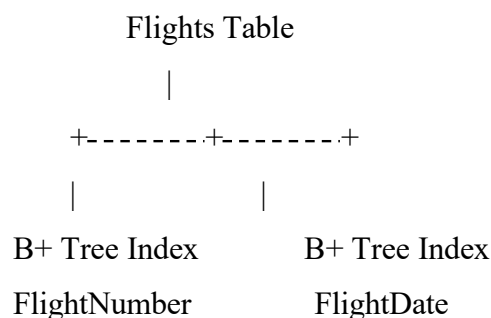
This is ideal for range queries:

```
SELECT *  
FROM Flights  
WHERE FlightDate BETWEEN '2026-08-01' AND '2026-08-31';
```

Because B+ tree leaf nodes are ordered and linked, the database can find the first matching date and scan the relevant range efficiently.

3. Combined structure

Conceptually:



Flight → Record Date → Records

This is preferable to using a single (FlightNumber, FlightDate) composite index because the **leading FlightNumber column makes the index excellent for flight-number lookups but less useful for date-only range queries.**

Performance

Operation	Index	Expected performance
Find one flight by number	B+ tree on FlightNumber	$O(\log N)$
Find flights in date range	B+ tree on FlightDate	$O(\log N + k)$
Insert/delete flight	Both indexes	$O(\log N)$

Here, **k** represents the number of flights returned by the range query.

Recommendation

Use **two complementary B+ tree indexes**:

FlightNumber → B+ Tree for fast point queries

FlightDate → B+ Tree for efficient range queries

This combined indexing strategy directly matches the two query patterns and avoids the limitations of trying to force both workloads into a single composite index.

7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

1. File Organization and Secondary Indexing

For a university student database, use a **heap file organization** for storing student records and **secondary B+ tree indexes** for department and CGPA.

1. File organization: Heap file

Store student records in an unordered heap file:

StudentID | Name | Department | CGPA | Year

1001 | Arun | CSE | 8.7 | 3

1002 | Ravi | ECE | 7.8 | 2

1003 | Meena | CSE | 9.1 | 4

...

Advantages:

- Fast insertion of new students.
- No need to physically reorder records.
- Simple record maintenance.
- Indexes handle efficient retrieval.

2. Secondary index on Department

Create a **B+ tree secondary index** on Department:

```
CREATE INDEX idx_department
ON Student(Department);
```

Since many students can belong to the same department, this is a **non-unique secondary index**.

For example:

```
SELECT *
FROM Student
WHERE Department = 'CSE';
```

The index quickly identifies the record pointers for CSE students instead of scanning the entire file.

3. Secondary index on CGPA

Create another **B+ tree index** on CGPA:

```
CREATE INDEX idx_cgpa
ON Student(CGPA);
```

B+ trees are especially suitable for range queries:

```
SELECT *
FROM Student
WHERE CGPA BETWEEN 8.0 AND 9.0;
```

The tree locates the starting CGPA and then follows the linked leaf nodes through the required range.

4. Composite index for combined queries

If queries frequently ask for students in a particular department **and** within a CGPA range:

```
SELECT *
FROM Student
WHERE Department = 'CSE'
AND CGPA BETWEEN 8.0 AND 9.0;
```

create a composite B+ tree index:

```
CREATE INDEX idx_dept_cgpa
```

ON Student(Department, CGPA);

The index first narrows the search to the **CSE** portion and then efficiently scans the **8.0–9.0 CGPA range**.

Recommended design

Component	Recommended choice	Purpose
Data file	Heap organization	Easy insertion and maintenance
Department index	Secondary B+ tree	Fast department searches
CGPA index	Secondary B+ tree	Efficient CGPA range queries
Combined queries	Composite B+ tree (Department, CGPA)	Fast department + CGPA filtering

Conclusion

The most suitable design is a **heap-organized student file with a composite secondary B+ tree index on (Department, CGPA)**. If department-only and CGPA-only queries are also very common, maintain separate indexes as well.

This provides efficient **department equality searches** and **CGPA range searches** while avoiding full-table scans.

8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Given Data

- Number of records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4,096 bytes**
- Assume **unspanned records**, meaning a record cannot cross a block boundary.

1. Records per block

$$\text{Records per block} = \lfloor \frac{4096}{50} \rfloor = 81$$

So each block stores **81 records**.

Space actually used for records:

$$81 \times 50 = 4050 \text{ bytes}$$

Unused space:

$$4096 - 4050 = 46 \text{ bytes}$$

Thus, block utilization is:

$$\frac{4050}{4096} \times 100 \approx 98.88\%$$

2. Number of blocks required

$$\lceil \frac{100000}{81} \rceil = 1235 \text{ blocks}$$

The final block contains:

$$100000 - (1234 \times 81) = 46 \text{ records}$$

So the final block uses:

$$46 \times 50 = 2300 \text{ bytes}$$

Overall data storage:

$$1235 \times 4096 = 5,058,560 \text{ bytes}$$

Actual record data:

$$100000 \times 50 = 5,000,000 \text{ bytes}$$

Overall utilization:

$$\frac{5,000,000}{5,058,560} \times 100 \approx 98.84\%$$

Comparison of File Organizations

Organization	Blocks	Utilization	Main characteristic
Heap	~1,235	~98.84%	Records stored wherever free space is available
Sorted	~1,235	~98.84%	Records physically ordered by a search key
Hashed	~1,235 minimum	~98.84% maximum	Records distributed among hash buckets

Heap organization

Heap storage requires approximately **1,235 blocks**. Since there is no ordering requirement, the blocks can be filled sequentially.

Advantage: Very good space utilization and fast insertion.

Disadvantage: Searching for a particular record may require scanning many blocks.

Sorted organization

The same **1,235 blocks** are required for the actual records, assuming no additional overflow space.

Advantage: Excellent for ordered and range queries, and binary search can reduce block accesses.

Disadvantage: Insertions and deletions can require expensive record movement or overflow handling.

Hashed organization

A hash file can also store the 100,000 records in approximately **1,235 blocks of actual data**, but practical implementations usually allocate **additional bucket/overflow space** to handle collisions.

For example, if we deliberately keep buckets only **80% full** to reduce overflow:

$$\frac{1,235}{0.8} \approx 1,544 \text{ blocks}$$

So a practical hashed organization might need around **1,544 blocks**, depending on the chosen load factor and collision strategy.

Final Analysis

All three organizations have essentially the same **theoretical data capacity** because record size and block size determine the number of records per block:

81 records/block

and

1,235 blocks for 100,000 records

The major difference is **not the basic record-storage efficiency**, but how much additional space is needed for ordering, hashing, overflow, and indexing.

- **Heap:** Best for simple storage and frequent insertions.
- **Sorted:** Best when range and ordered queries are frequent.
- **Hashed:** Best for fast equality searches, but may require extra overflow space.

For pure disk-block utilization, **Heap and Sorted are about 98.84% utilized**, while **Hashed utilization depends on its load factor and collision/overflow strategy**.

9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

1. Composite Indexing Strategy for 500 Million Posts

At **500 million posts**, a single large composite index is usually not the best solution because the platform has three different access patterns: user-based retrieval, time-based retrieval, and hashtag-based retrieval.

1. Primary index: user_id + timestamp

Create a composite **B+ tree index**:

```
CREATE INDEX idx_posts_user_time  
ON Posts(user_id, timestamp);
```

This should be the main index for queries such as:

```
SELECT *  
FROM Posts  
WHERE user_id = 12345  
ORDER BY timestamp DESC;
```

and:

```
SELECT *  
FROM Posts  
WHERE user_id = 12345  
AND timestamp BETWEEN '2026-08-01' AND '2026-08-25';
```

The ordering is important: `user_id` **comes first**, because it is the equality/filtering attribute, followed by timestamp for efficient range and ordering operations.

2. Hashtag index

Hashtags have a different access pattern, so create a separate index rather than extending the first composite index:

```
CREATE INDEX idx_posts_hashtag_time
```

```
ON Posts(hashtag, timestamp);
```

This supports queries such as:

```
SELECT *
```

```
FROM Posts
```

```
WHERE hashtag = '#sports'
```

```
AND timestamp BETWEEN '2026-08-01' AND '2026-08-25';
```

The timestamp component allows the database to efficiently retrieve only posts for that hashtag within the required time range.

3. Partition the 500 million posts

At this scale, indexing alone is insufficient. Partition the Posts table, preferably **by time**, such as monthly or weekly partitions:

Posts

|— 2026-06

|— 2026-07

|— 2026-08

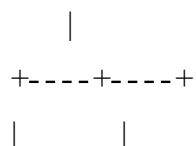
|— 2026-09

|— ...

Queries containing a timestamp range can then use **partition pruning**, avoiding unnecessary partitions.

Within each partition:

Posts Partition



`user_id,timestamp hashtag,timestamp`

B+ Tree

B+ Tree

4. Index choice

Query pattern	Recommended index
User's posts	B+ tree (user_id, timestamp)
User's posts in date range	B+ tree (user_id, timestamp)
Hashtag posts	B+ tree (hashtag, timestamp)
Hashtag within date range	B+ tree (hashtag, timestamp)
Global recent posts	Time-based partitioning + timestamp index if needed

5. Why not (user_id, timestamp, hashtag)?

A single index such as:

(user_id, timestamp, hashtag)

would be inefficient for hashtag-only queries because user_id is the leading column. A query searching for:

WHERE hashtag = '#sports'

cannot efficiently use the leading portions of that index.

Therefore, **two specialized composite indexes** are better than one three-column index.

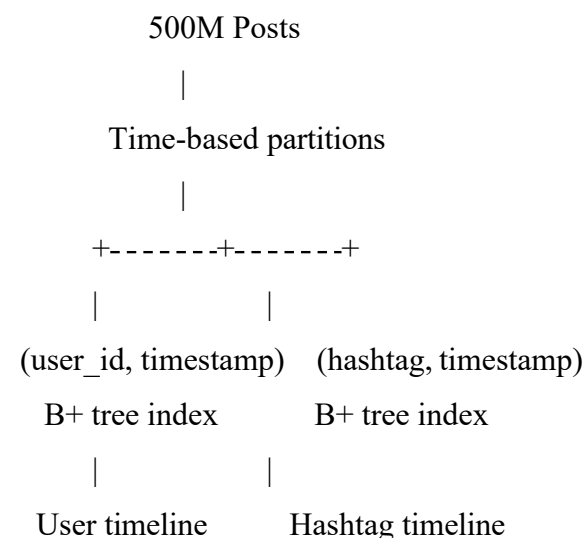
Scalability consideration

With **500 million records**, these indexes can themselves become very large. Partitioning by timestamp limits the amount of data and indexing work associated with each partition. In a distributed system, partitions can also be distributed across multiple database nodes.

For very high-volume hashtag searches, a dedicated **inverted index/search system** can complement the database B+ tree rather than forcing every hashtag query through the transactional database.

Final Design

The recommended architecture is:



Use **time partitioning** + **B+ tree** (user_id, timestamp) + **B+ tree** (hashtag, timestamp). This gives efficient user timelines, hashtag searches, and date-range queries while remaining scalable as the post volume continues to grow.

10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

1. Performance Comparison

For a supermarket billing system handling **1 million transactions per day**, assume the main search key is something like TransactionID. The three organizations have very different performance characteristics.

Operation	Heap File	Sorted File	Hashed File
Insert	O(1) average	O(n)	O(1) average
Search	O(n)	O(log n) with binary search	O(1) average
Delete	O(n)	O(n)	O(1) average
Range search	O(n)	Excellent	Poor
Maintenance	Very low	High	Moderate
Best use	Heavy inserts	Ordered/range queries	Exact-match queries

1. Heap File

Transactions are stored wherever free space is available.

- **Insert:** Very fast, approximately **O(1)**, because the new transaction can simply be appended.
- **Search:** **O(n)** because the system may have to scan all transactions.
- **Delete:** Usually **O(n)** if the transaction must first be located; after locating it, marking the space free is cheap.

For 1 million daily transactions, heap organization is attractive when transactions are **written much more frequently than they are searched**.

2. Sorted File

Transactions are maintained in sorted order, for example by TransactionID or TransactionDate.

- **Insert:** Expensive, approximately **O(n)**, because records may have to be shifted or placed in overflow areas.
- **Search:** Approximately **O(log n)** using binary search over blocks.

- **Delete:** Potentially **O(n)** because maintaining the sorted order may require reorganizing the file.
- **Range queries:** Excellent because records are physically ordered.

Thus, sorted files are useful for reports such as:

All transactions between 10:00 AM and 12:00 PM

but are less attractive when 1 million new transactions arrive every day.

3. Hashed File

Use a hash function such as:

$$h(\text{TransactionID}) = \text{TransactionID} \cdot B$$

where **B** is the number of buckets.

- **Insert: O(1) average**, assuming a good hash function and reasonable load factor.
- **Search: O(1) average** for exact TransactionID searches.
- **Delete: O(1) average**, after locating the bucket.
- **Range queries:** Poor because hashing does not preserve ordering.

Collision handling can use **overflow buckets or chaining**.

Best Choice

For a supermarket billing system, **Hashed File Organization is generally the best choice** if the dominant operation is retrieving individual bills using TransactionID.

With **1 million transactions/day**, the priorities are:

12. **Fast transaction insertion**
13. **Fast exact transaction lookup**
14. **Fast deletion/correction when required**

Hashing provides approximately **O(1) average time** for all three.

However, if the system frequently generates **date/time-based sales reports**, a practical design would be:

Hashed file for TransactionID lookups + a secondary B+ tree index on TransactionDate.

This combines fast point retrieval with efficient range/reporting queries.